Fundamentos da orientação a objetos. Modelo do processo unificado.

Analise e Modelagem de Sistemas

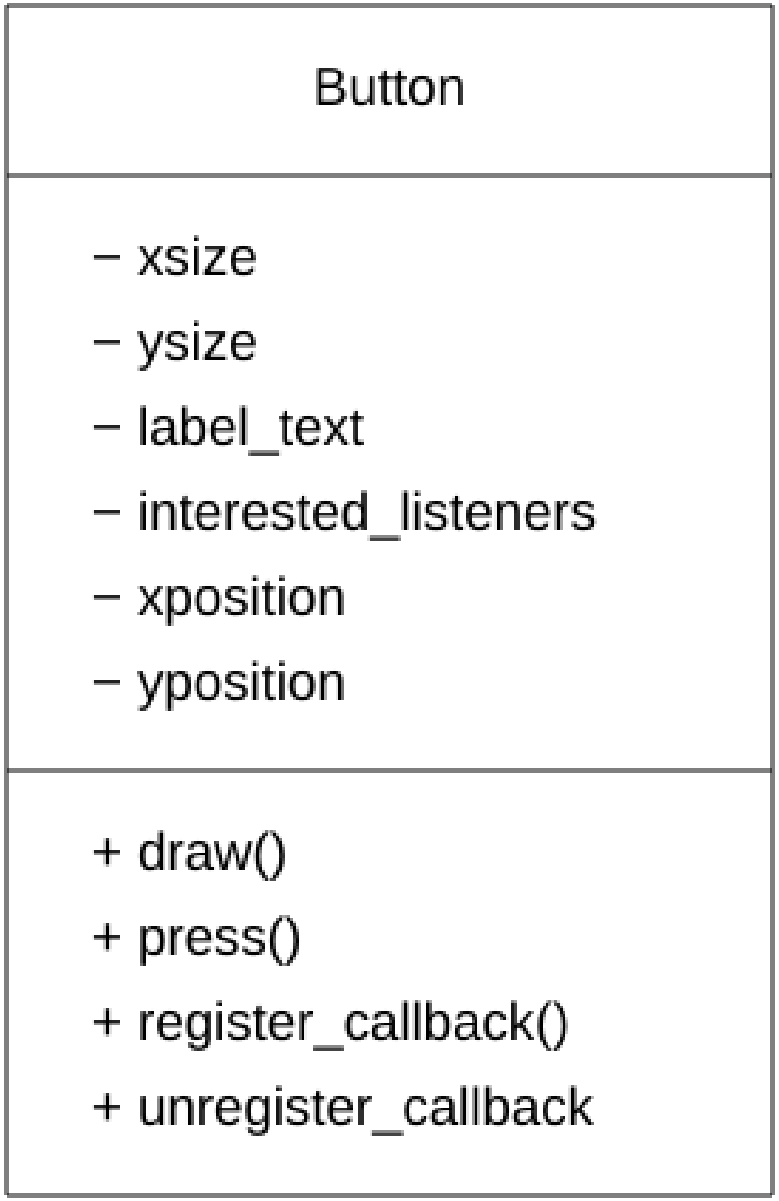
Eduardo Furlan Miranda

2025-10-29

Adaptado de: WERLICH, C. Análise e Modelagem de Sistemas. Londrina: EDE SA, 2020. ISBN 978-85-522-1683-4.

Orientação a objetos (OO)

- Paradigma baseado em objetos
- Um objeto é uma entidade que encapsula dados e funções
- Ex. de notação UML para uma classe:



OO

- 1966: linguagem de programação SIMULA
- Ex. em Python:

```
class Carro:
  def __init__(self, modelo):
    self.modelo = modelo
  def buzinar(self):
    print(f"{self.modelo}: Bi-bi!")

meu_carro = Carro("Fusca")
print(meu_carro.modelo)  # Acessa o atributo
meu_carro.buzinar()      # Chama o método
```

UML (1997)

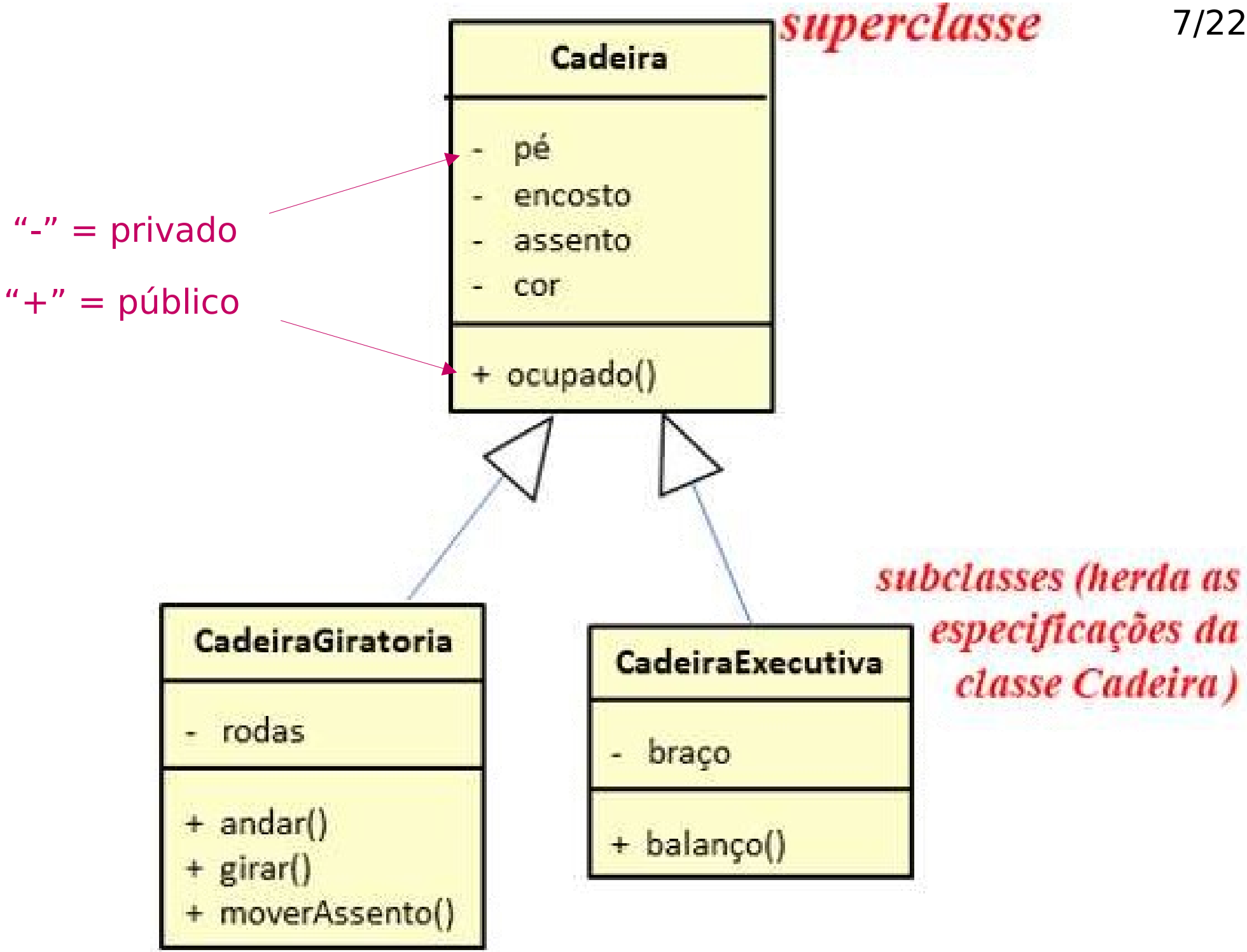
- Linguagem de modelagem de uso geral
  - Visual (usa coleção de modelos gráficos)
  - Orientada a objetos
- Padrão OMG, ISO/IEC 19501
- Usado em várias áreas além da computação
- Modela sistemas, independente de linguagens de programação
- Um modelo UML pode ser usado para gerar código fonte

OO - Abstração, classe, objeto

- Abstração como ideia central
- Classe: representação da abstração (“projeto”)
  - Representa um conjunto de objetos
    - Características (atributos) e comportamentos (métodos)
- Características: atributos (variáveis) e métodos (funções)
- Objeto: instância da classe, “prédio construído”
  - São únicos
  - Comportamentos: valores dos atributos e ações

OO - Herança, encapsulamento

- Herança
  - Novas classes reaproveitando o que já existe
- Superclasse e subclasses
  - A subclasse herda da superclasse, e pode acrescentar
- Encapsulamento
  - Esconde os detalhes da implementação
    - Ex.: não ter acesso direto às variáveis ou funções dentro do objeto


superclasse
Cadeira
- pé
- encosto
- assento
- cor
+ ocupado()
"-" = privado
"+" = público
CadeiraGiratoria
- rodas
+ andar()
+ girar()
+ moverAssento()
CadeiraExecutiva
- braço
+ balanço()
subclasses (herda as especificações da classe Cadeira)

OO - Polimorfismo estático

- O mesmo método é implementado várias vezes na mesma classe, com parâmetros diferentes
  - A escolha do método a ser chamado vai variar de acordo com o parâmetro passado

- Ex. Java:

```
class Calculadora {
    int somar(int a, int b) { return a + b; }
    int somar(int a, int b, int c) { return a + b + c; }
}
```

Obs.: Python não tem

OO - Polimorfismo dinâmico

- O mesmo método é implementado várias vezes nas subclasses derivadas, com os mesmos parâmetros
  - A escolha do método depende do objeto que o chama
    - E, consequentemente, da classe que o implementa

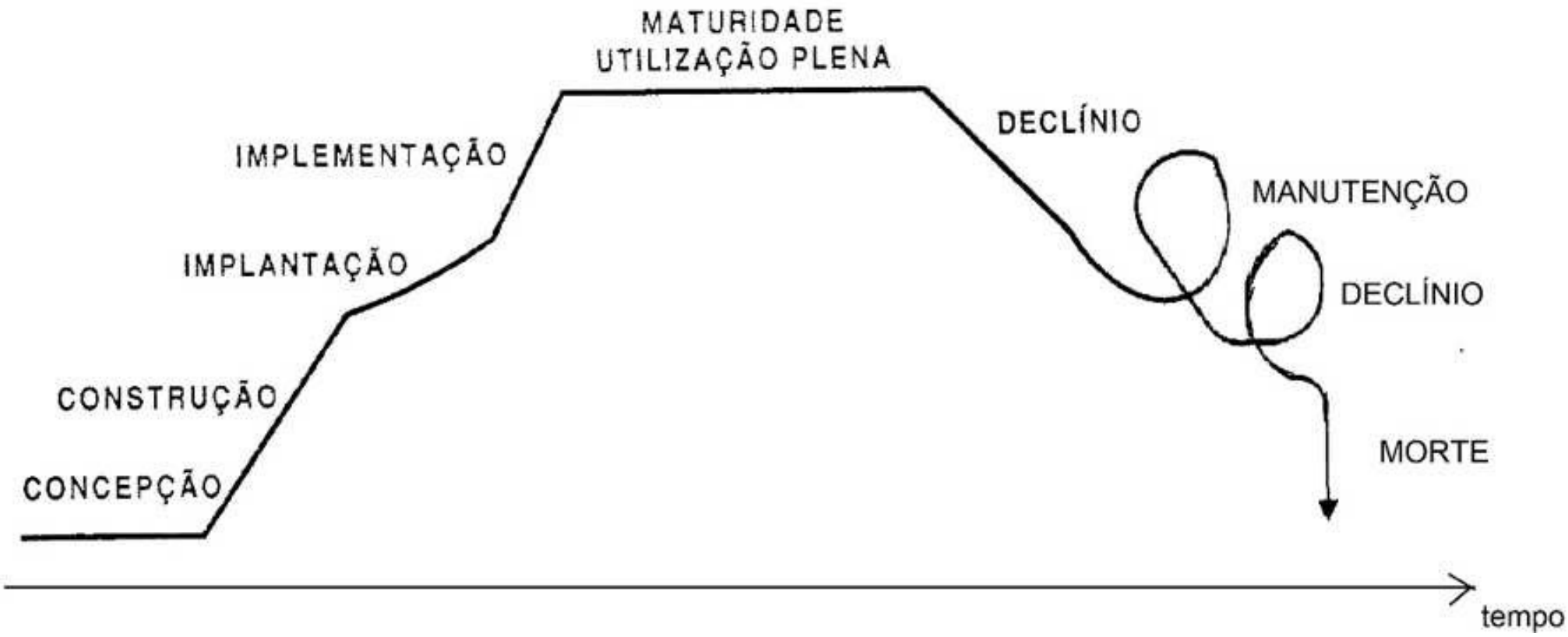
```
class Cachorro:
    def falar(self):
        print("Au au!")

class Gato:
    def falar(self):
        print("Miau!")

animais = [Cachorro(), Gato()]
for animal in animais:
```

OO - vantagens

- Reutilização de código
- Utilização de um único padrão conceitual para a análise, o projeto e a implementação
- O tempo de desenvolvimento do software é mais rápido
- A construção de sistemas mais complexos é simplificado pelo fato de cada objeto ser simples, fácil de testar e integrar ao demais objetos
- Pode ter os custos de desenvolvimento reduzidos

Analise e Modelagem de Sistemas

Ciclo de vida do software



Modelos

- Modelos que podem ser usados para organizar o ciclo de vida de desenvolvimento do software
  - Cascata
  - Em espiral
  - De prototipação
  - Incremental
  - Iterativo
  - dentre outros

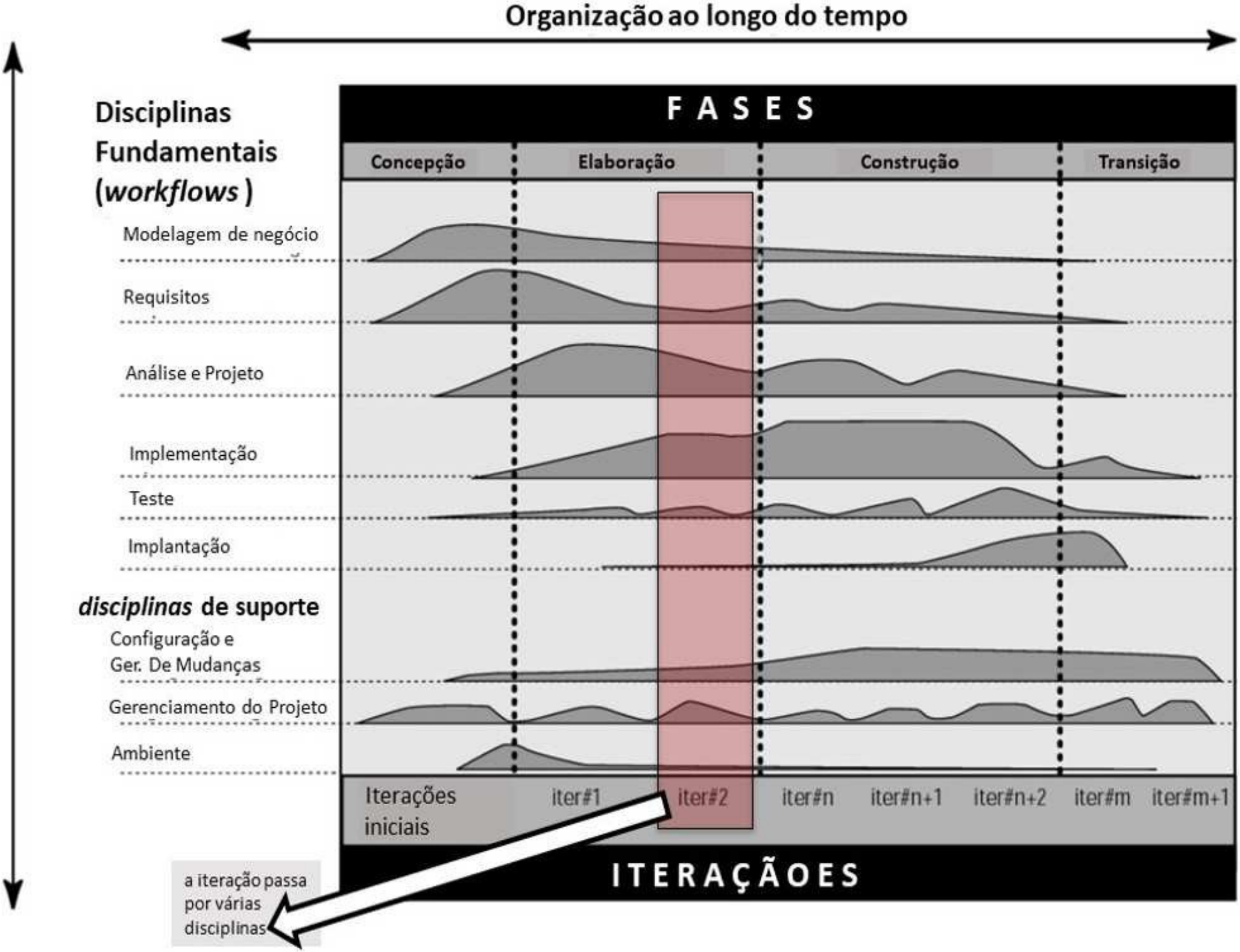
Modelos

- Cascata - consiste em 5 fases
  - Análise e levantamento de requisitos, projeto, desenvolvimento, teste, implantação
- Modelo iterativo (ou evolutivo)
  - Uma análise inicial e superficial é feita e o software implementado
  - Posteriormente, a análise é refinada e o software é melhorado
- Modelo incremental
  - Entregue em partes (módulos) ao cliente, sendo que cada módulo passa por todas as fases do modelo em cascata

Processo Unificado (PU)

- Modelo adaptativo, baseado no modelo incremental e que visa a construção iterativa de um software
- Aperfeiçoou o tradicional processo incremental e o iterativo, eliminando algumas desvantagens dos dois processos
- No modelo iterativo
  - Às vezes, o software não termina, pois o cliente está sempre solicitando alterações
  - Se o processo de documentação não for adequado, o software vira uma colcha de retalhos

- No modelo incremental
  - Cada parte tem de ser concluída integralmente para se passar à próxima
- No modelo PU
  - Os processos iterativos e incremental caminham em paralelo, permitindo que o software vá se tornando robusto a cada refinamento
- O PU surgiu depois da UML, é de uso público e contempla o ciclo de desenvolvimento do software


Organização ao longo do tempo
Disciplinas Fundamentais (*workflows*)
FASES
Concepção
Elaboração
Construção
Transição
Modelagem de negócio
Requisitos
Análise e Projeto
Implementação
Teste
Implantação
disciplinas de suporte
Configuração e Ger. De Mudanças
Gerenciamento do Projeto
Ambiente
Iterações iniciais
iter#1
iter#2
iter#n
iter#n+1
iter#n+2
iter#m
iter#m+1
ITERAÇÃOES
a iteração passa por várias disciplinas

Disciplinas

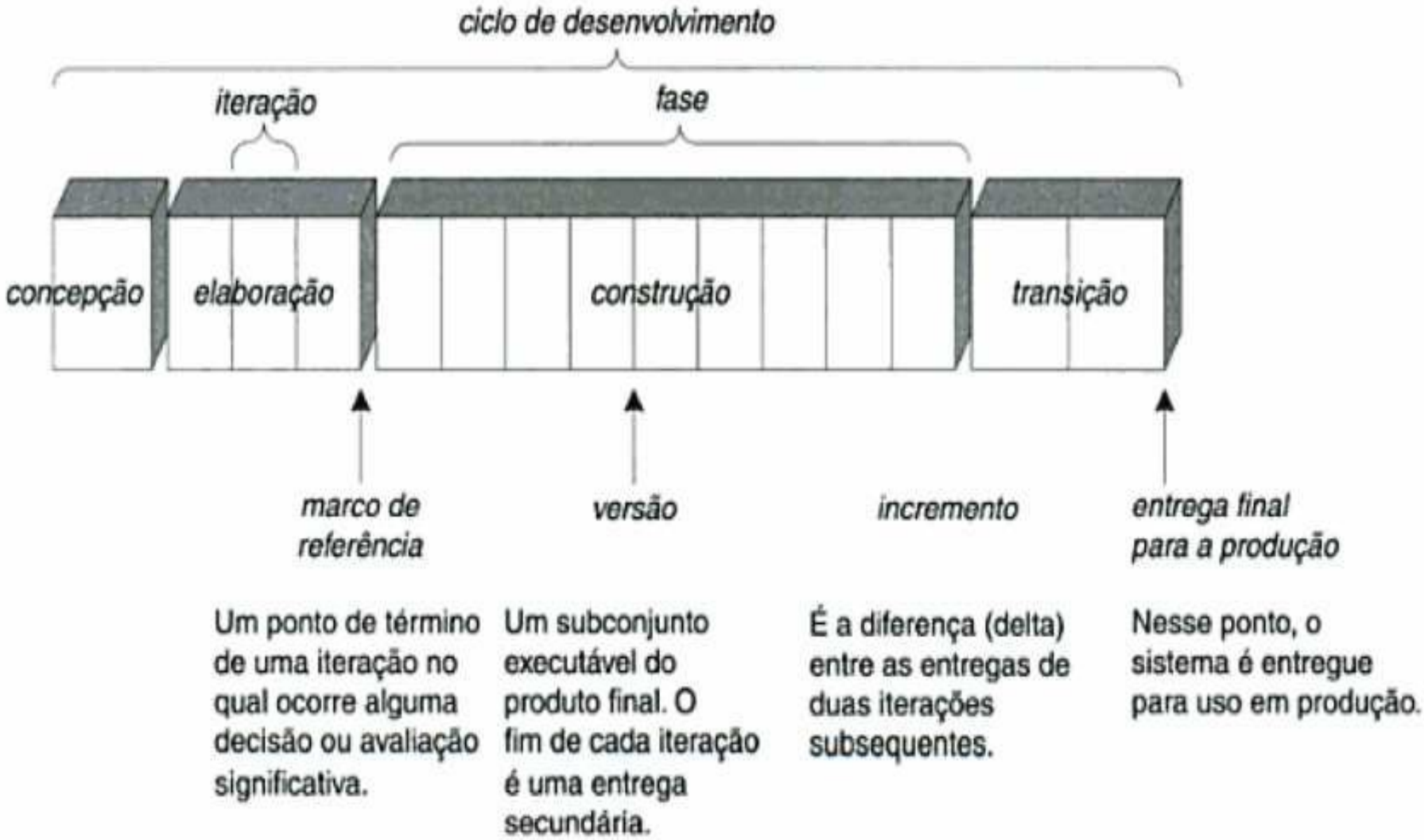
- Fases: modelagem de negócio, requisitos, análise e projeto, implementação, testes, e implantação

- As demais disciplinas estão associadas ao suporte e foram incluídas no Processo Unificado Racional (RUP)
  - Gerenciamento de mudanças e configurações
  - Gerenciamento de projeto e ambiente

Aspectos

- I. Dirigido por caso de uso
  - Responde o que cada usuário necessita
  - Avançando através de uma série de fluxos de trabalho (workflow)
- II. Centrado na arquitetura
  - Dá uma visão abrangente do sistema
  - Necessidades dos usuários e objetivos estratégicos da empresa
- III. Iterativo e incremental
  - A iteração consiste em dividir o projeto em subprojetos menores e o resultado de uma iteração produz um incremento
  - Reconhece que dificilmente em um sistema complexo, no início, é possível definir totalmente os requisitos e necessidades do usuário

Ciclo de vida do PU



Ciclo de vida do PU

- É uma série de repetições ao longo da vida do sistema, sendo que cada ciclo completo resulta em uma versão do software

- Cada ciclo é composto por 4 fases
  - Concepção: visão geral do projeto, o escopo e os requisitos iniciais
  - Elaboração: visão mais refinada dos requisitos e da arquitetura, análise de riscos e estimativas
  - Construção: desenvolvimento do sistema, começando pelos elementos mais fáceis, e inicia-se a preparação para a implantação
  - Transição: é a fase de implantação do sistema, ou seja, a entrega

Elementos básicos

- O PU descreve um processo por quatro elementos básicos:
  - Papel (worker, trabalhador): é a identificação das responsabilidades de cada indivíduo
  - Artefato: é o termo utilizado para identificar qualquer produto de trabalho que o worker gera
  - Atividade: é a tarefa executada pelo worker com o objetivo de produzir ou alterar um artefato
  - Disciplina (workflow – fluxo de trabalho): é a sequência de atividades que gera um resultado significativo
    - Estão associados a três perspectivas: dinâmica (tempo), estática (atividades) e boas práticas
    - Em cada uma encontramos um conjunto de disciplinas